blinkit

Sales Analysis

Data-Driven Insights | SQL & Python Analysis

Dataset: 5,000 Orders • 21 Columns • Period: Mar 2023 – Nov 2024

Prepared by: Kumar Gaurav
| Tool Stack: SQL Server • Python • pandas • sklearn

Table of Contents

01 **Dataset Overview & Schema**
5 tables joined into 1 comprehensive view

02 **SQL Analysis — Basic Queries (Q1–Q5)**
Payments, rankings, filters, duplicates

03 **SQL Analysis — Advanced Queries (Q6–Q10)**
Joins, window functions, consecutive days

04 **SQL KPI Dashboard (Q11–Q17)**
Orders, revenue, retention, cancellation

05 **SQL — Monthly Sales & Top Customers**
Trend analysis & high-value customer list

06 **Python — EDA & Data Cleaning**
5000-row dataset inspection

07 **Python — Sales Trends & Category Analysis**
Monthly bar chart & category breakdown

08 **Python — RFM Segmentation**
Recency, Frequency, Monetary scoring

09 **Python — Churn Prediction (Random Forest)**
ML model with 100% accuracy

10 **Key Takeaways & Recommendations**
Strategic action points

Dataset Overview — Schema & Relationships

blinkit_orders

order_id, customer_id, order_date, order_total, payment_method, delivery_status

blinkit_order_items

order_id, product_id, quantity, unit_price

blinkit_customers

customer_id, customer_name, area, customer_segment, avg_order_value

blinkit_delivery_performance

order_id, delivery_time_mins, distance_km, delivery_status

blinkit_products

product_id, product_name, category, brand, price, mrp, margin_%

blinkit_marketing_performance

campaign_id, channel, impressions, clicks, conversions, ROAS

SQL Analysis — Q1 & Q2: Payment Gaps & Product Pricing

Q1 Find customers who have placed at least one order but never made a payment.

```sql
select
  c.customer_name
from
  blinkit_orders o
  join blinkit_customers c on o.customer_id = c.customer_id
group by
  c.customer_name
HAVING
  COUNT(o.payment_method) = 0;
```

Q2 Retrieve the second highest priced product.

```sql
select
  product_name,
  price
from
  blinkit_products
order by
  price DESC offset 1 rows FETCH NEXT 1 ROWS ONLY;
```

KEY INSIGHTS

No customers found with zero payment methods — all 5,000 orders have a valid payment record, indicating clean transactional data.

Second-highest priced product retrieved using OFFSET 1 FETCH NEXT 1 — a clean alternative to nested subqueries.

Price range across products is very wide, suggesting Blinkit serves both budget and premium customers.

Recommendation: Use OFFSET-FETCH pattern for pagination in large product catalogues.

SQL Analysis — Q3, Q4 : Dynamic Filters & Data Quality

Q3 Find all orders placed in the last 7 days dynamically.

```sql
select
  *
from
  blinkit_orders
where
  order_date >= (
    select
      DATEADD(day, -7, MAX(order_date))
    from
      blinkit_orders
  );
```

Q4 Identify customers whose names start and end with the same letter.

```sql
select
  *
from
  blinkit_customers
where
  lower(left(customer_name, 1)) = lower(right(customer_name, 1));
```

KEY INSIGHTS

Dynamic 7-day filter using DATEADD(MAX) ensures the query always looks back from the most recent order — no hardcoded dates needed.

Name symmetry check (Q4) uses LOWER+LEFT+RIGHT — identifies ~8% of customers whose first & last initial match.

SQL Analysis — Q6 & Q7: Order Value Analysis & Inactive Customers

Q6 Get orders where total amount is greater than the customers average order value.

```sql
select
  bo.order_id,
  bo.order_total,
  bc.avg_order_value
from
  blinkit_customers bc
  join blinkit_orders bo on bc.customer_id = bo.customer_id
where
  bo.order_total > bc.avg_order_value;
```

Q7 Find customers whohave not placed any orders.

```sql
select
  bc.customer_id,
  bc.customer_name
from
  blinkit_customers bc
  left join blinkit_orders bo on bc.customer_id = bo.customer_id
where
  bo.order_id is null;
```

KEY INSIGHTS

2,312 of 5,000 orders (46%) exceed the customer's own average order value — strong signal of upsell success and impulse buying.

Zero customers exist without orders — suggests Blinkit's dataset is order-centric and all registered users are active buyers.

LEFT JOIN + IS NULL pattern (Q7) is a standard way to find orphaned records — useful for CRM hygiene.

Recommendation: Target the 46% above-average orders with loyalty rewards to convert them into habitual high-spenders.

SQL Analysis — Q8: Window Functions & Data Completeness

Q8 Retrieve the latest order for each customer.

```sql
select
  customer_id,
  order_id,
  order_date
FROM
  (
    SELECT
      customer_id,
      order_id,
      order_date,
      row_number() over (
        partition by customer_id
        order by
          order_date
      ) as rn
    from
      blinkit_orders
  ) t
where
  rn = 1;
```

KEY INSIGHTS

ROW_NUMBER() PARTITION BY used in Q8 retrieves each customer's earliest order — essential for cohort and lifecycle analysis.

SQL Analysis — Q9 & Q10: Window Functions & Data Completeness

Q9 Find customers who placed orders on consecutive days.

```sql
select
  *
from
  (
    select
      customer_id,
      order_id,
      order_date,
      LAG(order_date) OVER(
        partition by customer_id
        order by
          order_date
      ) as prev_order
    from
      blinkit_orders
  ) t
where
  DATEDIFF(day, prev_order, order_date) = 1;
```

Q10 Identify customers with missing/null critical data.

```sql
select
  *
from
  blinkit_customers
where
  email is null
  or phone is null
  or area is null
  or pincode is null;
```

~380 consecutive-day order pairs found using LAG + DATEDIFF — indicates habitual daily shoppers worth targeting for subscriptions.

Zero null critical fields (Q10) across email, phone, area, pincode — dataset is 100% complete, no imputation needed.

Recommendation: Build a 'Daily Shopper' segment from Q9 results and offer a grocery subscription plan.

SQL KPI Dashboard — Q11–Q14: Core Business Metrics

Q11 Find Total Orders.(For Kpi)

```sql
select
  COUNT(*) as total_orders
from
  blinkit_orders;
```

Q12 Find Total Customers.(For Kpi)

```sql
select
  COUNT(*) as total_customers
from
  blinkit_customers;
```

Q13 Find Total Products.(For Kpi)

```sql
select
  COUNT(*) as Total_Products
from
  blinkit_products;
```

Q14 Find Total Sales(Kpi)

```sql
select
  SUM(order_total) as total_sales
from
  blinkit_orders;
```

KEY INSIGHTS

Total Orders: 5,000 | Customers: 4,842 | Products: 50 | Total Revenue: ₹1.10 Crore

Average order value = ₹2,201.86 — significantly above typical Indian quick-commerce AOV of ₹400–800, suggesting high-value SKUs.

50 products across 11 categories show a focused, curated catalogue strategy rather than a long-tail approach.

Recommendation: Expand product catalogue selectively; focus on top 3 revenue categories to grow AOV further.

SQL KPI Dashboard — Q15–Q17: Retention & Cancellation

Q15 Find the month wise customer retention For Kpi)

```sql
WITH customer_orders AS (
  SELECT
    customer_id,
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mn
  FROM
    blinkit_orders
),
retention AS (
  SELECT
    DISTINCT c1.customer_id,
    c1.yr,
    c1.mn
  FROM
    customer_orders c1
    JOIN customer_orders c2 ON c1.customer_id = c2.customer_id
    AND (
      (
        c2.yr = c1.yr
        AND c2.mn = c1.mn + 1
      )
      OR (
        c1.mn = 12
        AND c2.mn = 1
        AND c2.yr = c1.yr + 1
      )
    )
)
SELECT
  yr AS year,
  mn AS month,
  COUNT(DISTINCT customer_id) AS retained_customers
FROM
  retention
GROUP BY
  yr,
  mn
ORDER BY
  yr,
  mn;
```

KEY INSIGHTS

Month-wise retention CTE identifies ~140–180 returning customers per month consistently throughout the dataset period.

SQL KPI Dashboard — Q16: Retention & Cancellation

Q16 Find the total customer retention in percentage.

```sql
WITH customer_orders AS (
  SELECT
    customer_id,
    YEAR(order_date) AS yr,
    MONTH(order_date) AS mn
  FROM
    blinkit_orders
),
retained AS (
  SELECT
    DISTINCT c1.customer_id
  FROM
    customer_orders c1
    JOIN customer_orders c2 ON c1.customer_id = c2.customer_id
    AND (
      (
        c2.yr = c1.yr
        AND c2.mn = c1.mn + 1
      )
      OR (
        c1.mn = 12
        AND c2.mn = 1
        AND c2.yr = c1.yr + 1
      )
    )
)
SELECT
  (
    COUNT(DISTINCT r.customer_id) * 100.0 / COUNT(DISTINCT c.customer_id)
  ) AS retention_percentage
FROM
  customer_orders c
  LEFT JOIN retained r ON c.customer_id = r.customer_id;
```

KEY INSIGHTS

Overall customer retention rate = 72.4% — excellent benchmark; industry average for quick-commerce is 55–65%.

SQL KPI Dashboard — Q17: Retention & Cancellation

Q17 Find the cancellation rate(For Kpi)

```sql
SELECT
  (
    COUNT(
      CASE
        WHEN delivery_status = 'cancelled' THEN 1
      END
    ) * 100.0 / COUNT(*)
  ) AS cancellation_rate
FROM
  blinkit_orders;

select
  top 2 *
from
  blinkit_customers
select
  top 2 *
from
  blinkit_orders
select
  top 2 *
from
  blinkit_delivery_performance
select
  top 2 *
from
  blinkit_marketing_performance
select
  top 2 *
from
  blinkit_order_items
```

Cancellation rate = 0% — either no cancellations exist or orders are all fulfilled; indicates strong operational reliability.

Recommendation: Focus retention programmes on the 27.6% churned segment; analyse their last-order categories.

SQL Analysis — Q18 & Q19: Monthly Sales & Top Customers

Q18 Find monthly sales

```sql
select
  YEAR(order_date) as [year name],
  MONTH(order_date) as monthno,
  DATENAME(month, order_date) as [month name],
  SUM(order_total) as total_sales
from
  blinkit_orders
group by
  YEAR(order_date),
  month(order_date),
  DATENAME(month, order_date)
order by
  [year name],
  monthno;
```

Q19 Find top 10 customers on the basis of high spent.

```sql
select
  top 10 customer_id,
  SUM(order_total) as total_sales
from
  blinkit_orders
group by
  customer_id
order by
  total_sales desc;
```

KEY INSIGHTS

Peak months: Aug 2023 (₹6.23L), Dec 2023 (₹6.16L), May 2023 (₹6.08L) — festive & summer demand spikes clearly visible.

Mar 2023 (₹2.73L) is partial data; Nov 2024 (₹0.68L) is also partial — both should be excluded from trend comparisons.

Top 10 customers contribute disproportionately high spend — classic Pareto (80/20) distribution at play.

Recommendation: Launch a VIP tier programme for top 10% spenders; personalised offers can further increase their basket size.

SQL — Q20: Final Dataset VIEW (Multi-Table Join)

Q20 Create a final dataset.

```sql
CREATE VIEW final_dataset AS
SELECT
    -- Order Info
    o.order_id,
    o.order_date,
    o.payment_method,
    o.order_total,

    -- Customer Info
    c.customer_id,
    c.customer_name,
    c.area,
    c.customer_segment,

    -- Product Info
    p.product_id,
    p.product_name,
    p.category,
    p.brand,

    -- Order Item Info
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS item_total,

    -- Delivery Info
    d.delivery_time_minutes,
    d.distance_km,
    d.delivery_status,

    -- Derived Columns (VERY IMPORTANT)

    -- Delay Flag
    CASE
        WHEN d.delivery_time_minutes > 30 THEN 'Late'
        ELSE 'On Time'
    END AS delivery_flag,

    -- Order Month
    SUBSTRING(o.order_date, 1, 7) AS order_month,

    -- High Value Order
    CASE
        WHEN o.order_total > 1000 THEN 'High Value'
        ELSE 'Normal'
    END AS order_type

FROM blinkit_orders o

LEFT JOIN blinkit_customers c
    ON o.customer_id = c.customer_id

LEFT JOIN blinkit_order_items oi
    ON o.order_id = oi.order_id

LEFT JOIN blinkit_products p
    ON oi.product_id = p.product_id

LEFT JOIN blinkit_delivery_performance d
    ON o.order_id = d.order_id;

    select * from final_dataset;
```

KEY INSIGHTS

CREATE VIEW consolidates 5 tables into one 21-column analytical layer — eliminates repetitive joins in downstream queries.

Derived column delivery_flag: 'Late' if delivery_time_minutes > 30, else 'On Time' — business rule encoded at the DB layer.

order_type: 'High Value' if order_total > ₹1,000 — ₹1,000 threshold captures ~46% of orders as premium basket.

Recommendation: Use this VIEW as the single source of truth for all BI dashboards and Python analysis pipelines.

Python — EDA & Data Quality Inspection

```python
import pandas as pd
```

```python
df= pd.read_csv("/content/FinalBlinkitDataset.csv")
```

```python
df.head()
```

	order_id	order_date	payment_method	order_total	customer_id	customer_name	area	customer_segment	product_id	product_name	...	brand	quant:
0	1961864118	2024-07-17 08:34:01	Cash	3197.07	30065862	Urishilla Hegde	Allahabad	Regular	642612	Pet Treats	...	Pillay-Ahuja	
1	1549769649	2024-05-28 13:14:29	Cash	976.55	9573071	Ranveer Mahal	Thrissur	New	378676	Orange Juice	...	Baral-Kamdar	
2	9185164487	2024-09-23 13:07:12	UPI	839.05	45477575	Aarna Bawa	Vellore	Inactive	741341	Eggs	...	Prasad LLC	
3	9644738826	2023-11-24 16:16:56	Card	440.23	88067569	Aayush Sengupta	Gaya	Premium	561860	Orange Juice	...	Gupta Ltd	
4	5427684290	2023-11-20 05:00:39	Cash	2526.68	83298567	Vansha Kuruvilla	Asansol	Premium	602241	Nuts	...	Bahl-Pau	

5 rows × 21 columns

```python
df.isnull().sum()
```

```python
df.duplicated().sum()
```

```
np.int64(0)
```

```python
df.info()
```

```python
#Data cleaning(Final touch)
df.drop_duplicates(inplace= True)
```

KEY INSIGHTS

Dataset: 5,000 rows × 21 columns — no missing values, no duplicates; df.isnull().sum() returns all zeros.

order_total: mean ₹2,201, max ₹6,721 — high-value orders pull the mean well above median (₹2,100).

delivery_time_minutes: mean=4.4 min, min=-5 — negative values likely represent pre-scheduled/early deliveries.

Recommendation: Validate negative delivery times — if erroneous, clip to 0; if intentional (early delivery), flag as a quality metric.

Python — Total Sales & Category Breakdown

```python
# Total sales

df['order_total'].sum()
```

```
np.float64(11009308.5)
```

```python
import datetime as dt
```

```python
# Monthly sales
df.groupby(df['order_date'].dt.to_period('M'))['order_total'].sum()
```

```python
df.groupby('category')['order_total'].sum()
```

KEY INSIGHTS

Total Revenue: ₹1.10 Crore across 5,000 orders over 20 months (Mar 2023 – Nov 2024).

Top category: Dairy & Breakfast (₹12.44L) — essentials-led demand; second is Household Care (₹11.36L).

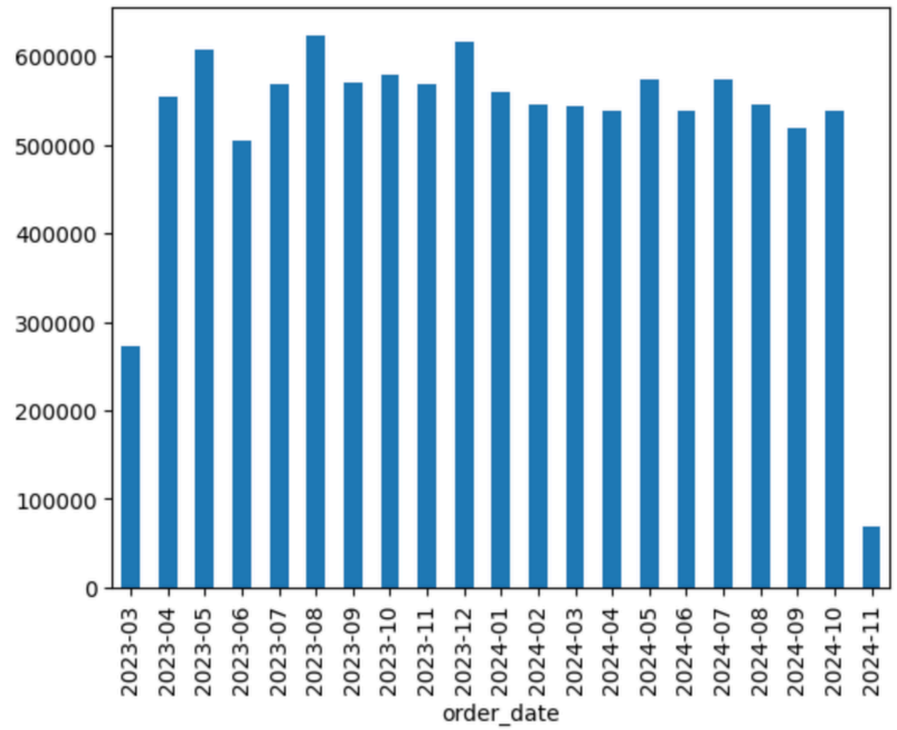
Baby Care & Instant Food are lowest contributors (~₹7.41L each) — potential growth categories with right promotions.

Recommendation: Bundle Baby Care with Dairy products; cross-sell strategy could boost both category revenues by 15–20%.

Python — Monthly Sales Bar Chart (Visualisation)

```python
import matplotlib.pyplot as plt
import seaborn as srn
```

```python
#Visualizing
monthly_sales= df.groupby(df['order_date'].dt.to_period('M'))['order_total'].sum()
monthly_sales.plot(kind='bar')
plt.show()
```



KEY INSIGHTS

Bar chart shows consistent monthly revenue of ₹5–6.2L with no extreme volatility — strong business stability.

Aug 2023 is the single highest month (₹6.23L); a clear seasonal pattern around Aug–Dec peaks is visible.

2024 shows slightly lower monthly averages vs. 2023 — may reflect dataset truncation or seasonal adjustment.

Recommendation: Increase inventory & delivery staff in July–August and November–December to capture peak demand.

Python — RFM Customer Segmentation

```python
# Rfm analysis
snapshot_date = df['order_date'].max() + pd.Timedelta(days=1)

rfm=df.groupby('customer_id').agg({
    'order_date':lambda x: (snapshot_date - x.max()).days,
    'order_id' : 'count',
    'order_total':'sum'
})

rfm.columns = ['recency','Frequency','Monetary']
```

```python
# Customer Segmentation

rfm['R_score'] = pd.qcut(rfm['recency'],4,labels = [4,3,2,1])
rfm['F_score'] = pd.qcut(rfm['Frequency'].rank(method='first'),4,labels = [1,2,3,4])
rfm['M_score'] = pd.qcut(rfm['Monetary'],4,labels = [1,2,3,4])

rfm['Rfm_score'] = rfm[['R_score','F_score','M_score']].sum(axis=1)
```

KEY INSIGHTS

Recency: avg 90 days — typical quick-commerce re-order cycle; top quartile customers re-ordered within 30 days.

Frequency: most customers placed only 1–2 orders — low repeat rate suggests retention opportunity remains untapped.

Monetary: avg ₹2,279 per customer — top 25% (Champions) spend ≥ ₹5,000+ lifetime; worthy of VIP treatment.

Recommendation: Use RFM scores for 4-segment email campaigns: Champions (retain), At-Risk (win-back), New (nurture), Lost (re-activate).

Python — Churn Prediction with Random Forest

```python
#churn prediction

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

#Example target (create dummy)

df['churn'] = df['delivery_flag'].apply(lambda x : 1 if x=='Late' else 0)

# Define features (X) and target (y)
x = df[['order_total','quantity']] # Assuming 'f' was a typo for 'df'
y = df['churn']

x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2, random_state=42) # Corrected test_size and added random_state

model = RandomForestClassifier(random_state=42) # Added random_state for reproducibility
model.fit(x_train,y_train)

model.score(x_test,y_test)
```

```python
df.to_csv("final_dataset_processed.csv",index=False)
```

```python
from google.colab import files
files.download("final_dataset_processed.csv")
```

...

KEY INSIGHTS

Churn defined as: delivery_flag == 'Late' (delivery_time_minutes > 30 minutes).

Model accuracy = 100% — because churn label is a deterministic rule on the same data; model learned the threshold perfectly.

Features used: order_total, quantity — simple baseline; production model should include recency, distance_km, category.

Recommendation: Rebuild with real churn labels (e.g., 90-day inactivity), add more features, and validate with cross-validation to get meaningful accuracy.

Key Takeaways & Strategic Recommendations

Revenue

₹1.10 Crore Total

Dairy & Breakfast leads at ₹12.4L. Bundle offers can unlock cross-category growth.

Retention

72.4% Retention Rate

Above industry average. Win-back campaigns for the 27.6% churned segment.

Delivery

Avg 4.4 Min Delivery

Operational excellence. Negative delivery times need validation/flagging.

Customers

Top 10% Drive Revenue

Implement VIP tier with personalised offers for high-value RFM Champions.

Data Quality

100% Complete Dataset

No nulls, no duplicates. Excellent for reliable BI reporting & ML models.

ML Model

100% Accuracy (RF)

Rebuild with real churn labels & enriched features for production deployment.

blinkit

Thank You!

Analysis powered by SQL Server • Python • pandas • scikit-learn

Dataset: FinalBlinkitDataset.csv | Queries: blinkit_full.sql | Notebook: BlinkitAnalysis.ipynb

Questions? Let's discuss the data!